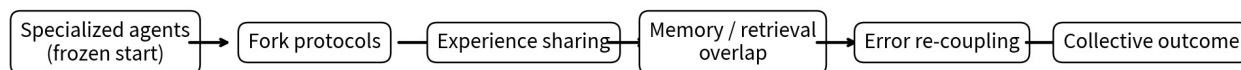


第 15 讲：经验分享会把 Agent 重新变得一样吗？

Experience Sharing, Heterogeneity Stability and Collapse

从“长期经历形成异质性”继续到“共享经验是否造成长期 Diversity Collapse”

第15讲：Experience Sharing → Heterogeneity Stability / Collapse



关键问题：分享带来的 transfer benefit，是否超过 specialization erosion + correlation growth？

图 1 第 15 讲的机制链：共享经验既可能带来跨域迁移，也可能让错误重新耦合。

1. 从第 14 讲继续：异质性形成以后，系统才真正开始危险

第 14 讲研究的是异质性“怎样长出来”。第 15 讲研究的是更像真实组织的问题：已经有不同专长的 Agent，如果开始持续交换经验，会发生什么？直觉上，共享经验应该让大家互相学习；但从系统角度看，共享也可能让原本互补的 Agent 逐渐读取同样的记忆、采用同样的判断捷径，并在同一批困难样本上一起犯错。

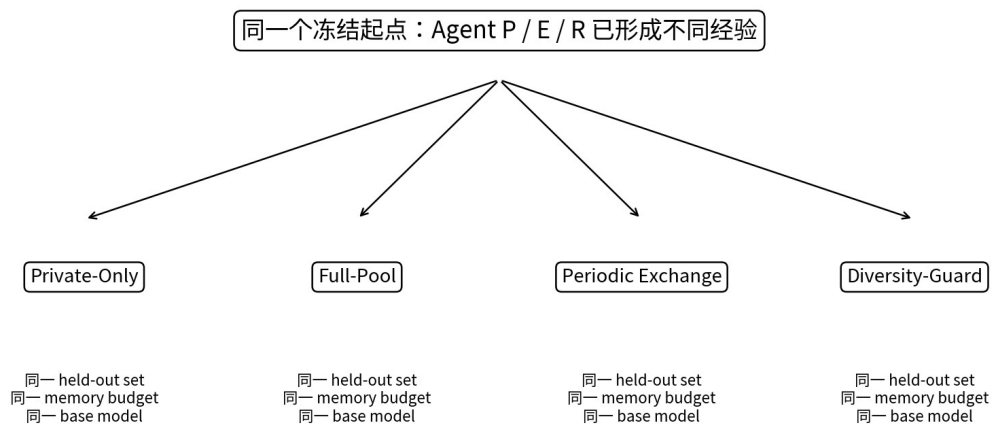
- 正向路径：Experience Sharing → Cross-domain Transfer → individual improvement.
- 负向路径：Experience Sharing → Memory Overlap → Retrieval Overlap → Behavioral Convergence → Error Re-coupling.
- 真正的问题不是“共享好不好”，而是 transfer benefit 是否大于 specialization erosion 与 correlation cost。

核心科研问题

当 Persistent Agents 已经由不同长期经历形成了功能性异质性以后，什么样的 memory sharing 能扩散有价值经验，同时又不抹平错误互补性？

2. 为什么不能直接比较“分享前 vs 分享后”？

因为时间继续前进时，Agent 自己也会继续获得新经验。若你让分享组继续学习，而不分享组停在原地，任何差异都无法归因于 sharing。正确做法是先冻结一个共同起点，再从同一状态 fork 出多个 protocol。



只有 sharing policy 改变 → 才能讨论因果归因

图2 同一起点 fork：只改变 sharing protocol，其余变量保持一致。

- 冻结同一 checkpoint 的三个 local memory bank。
- 冻结 base model、system prompt、role、tool、temperature、retrieval top-k 与 held-out evaluation。
- 所有 protocol 使用相同 memory budget；不能让 Full-Pool 因为上下文更长而天然占便宜。
- 每一轮只改变“哪些 experience 能进入谁的 memory”。

3. 四个最小对照条件

条件	分享规则	主要回答的问题
C0 Private-Only	不分享；各自保留 local memory	专业化在没有交换时是否稳定？
C1 Full-Pool	所有 Agent 的经验进入共同池	最大信息共享是否导致同质化/共错？
C2 Periodic Exchange	每轮只交换部分 peer experience	稀疏交换能否减缓 collapse？
C3 Diversity-Guard	只分享高质量、跨专长、带 provenance 的新颖经验，并保留 local quota	能否获得 transfer 而不抹平 specialization？

4. 先定义“长期 Diversity Collapse”

第8讲的 diversity collapse 发生在一次讨论中；第15讲研究的是跨 episode、跨 session 的长期版本。我们至少同时看四层变化：memory、retrieval、behavior、error。

层	指标示例	危险信号
Memory	memory overlap / Jaccard	三个库越来越相同
Retrieval	同一 case 检索到相同 experience 的比例	表面记忆不同，但实际用到的证据相同
Behavior	prediction agreement / specialization index	三人行为越来越同质
Error	pairwise error ϕ / wrong consensus	最危险：一起错的概率上升

5. 两个必须同时报告的效应：Transfer 与 Erosion

共享经验的价值不能只用平均准确率概括。对每个 Agent，要分开看 specialty family 和 off-specialty family。

Transfer Benefit = OffSpecialtyError_before - OffSpecialtyError_after
Specialty Erosion = SpecialtyError_after - SpecialtyError_before
Error Re-coupling = phi_after - phi_before
Collective Gain = AvgMemberError - MajorityError

理想协议应该让 Transfer Benefit > 0、Specialty Erosion 接近 0、Error Re-coupling 接近 0 或更低，同时保持正的 Collective Gain。

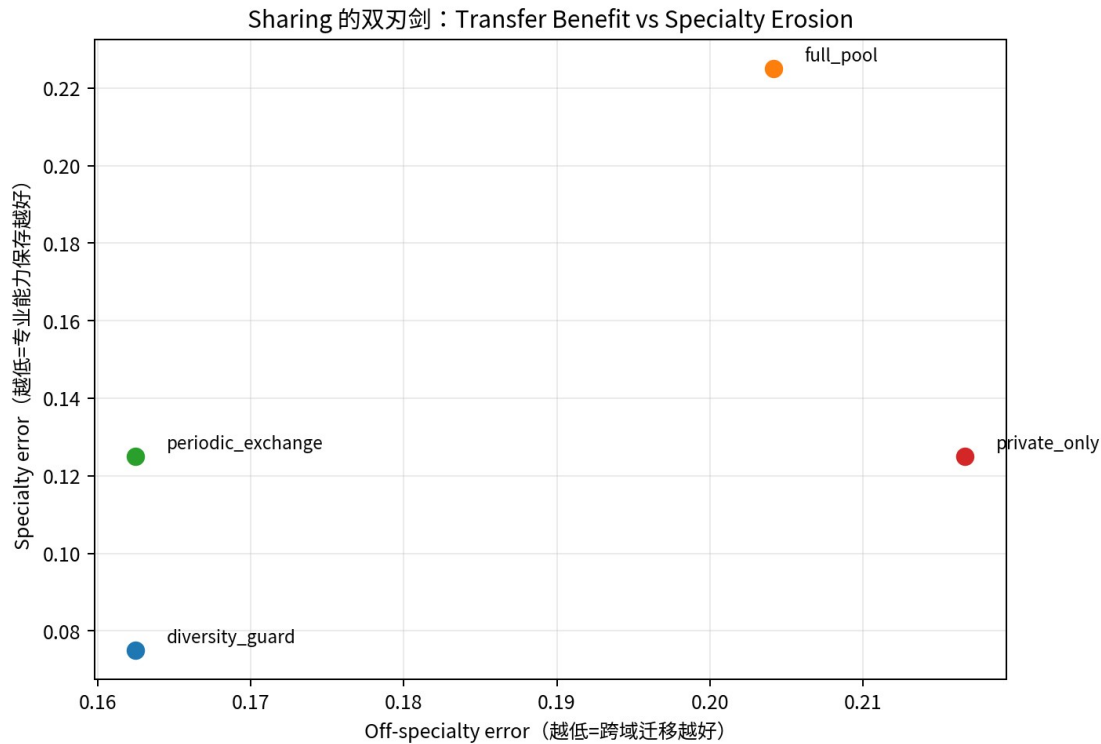


图 3 Teaching Mock 中 round 4 的“跨域迁移 vs 专长侵蚀”示意。

6. Teaching Mock：为什么 Full-Pool 可能越分享越糟

Protocol	成员错误	Specialization	Error ϕ	Majority error	Wrong consensus
private_only	18.6%	0.092	0.077	10.0%	1.7%
full_pool	21.1%	-0.021	0.423	14.2%	10.0%
periodic_exchange	15.0%	0.037	0.109	5.8%	2.5%
diversity_guard	13.3%	0.087	0.163	5.8%	2.5%

这组数字完全是受控 Teaching Mock，不代表任何真实模型。Mock 刻意让 Full-Pool 的 off-specialty 能力有所改善，但共享 latent error 也越来越强，因此 memory overlap 和 error correlation 上升，最终 collective gain 可能被侵蚀。

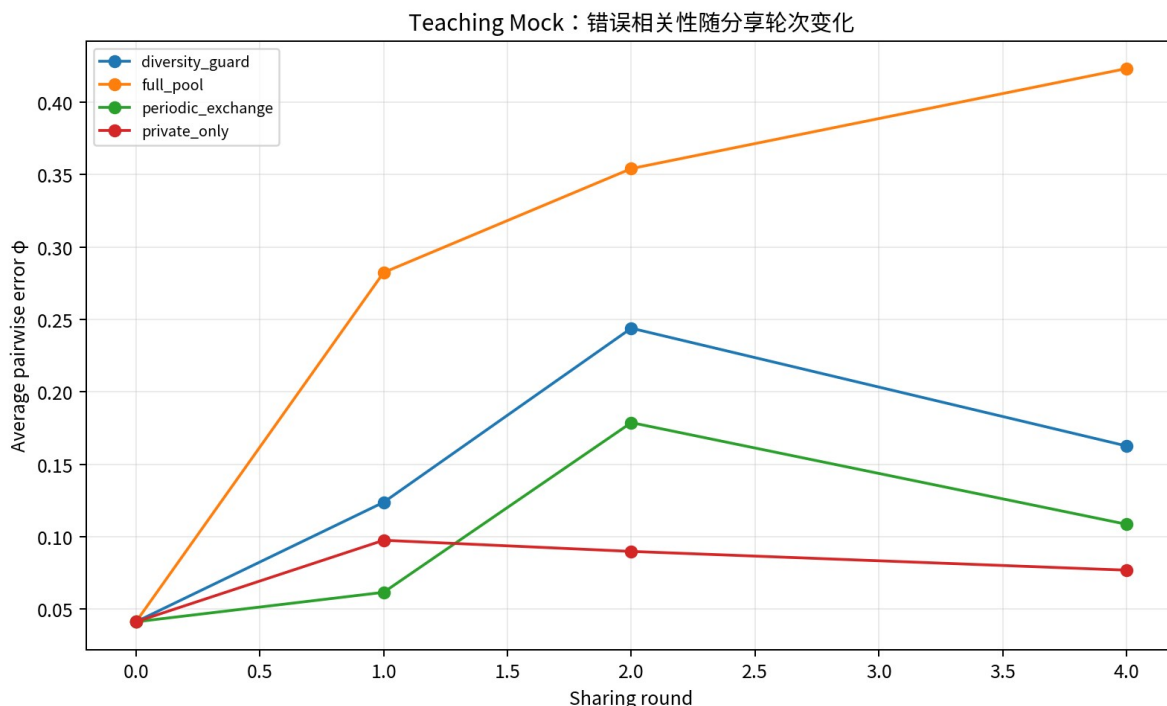


图4 Teaching Mock：分享轮次越多，Full-Pool 的错误相关性增长最快。

7. 为什么“大家都学到了更多”仍可能让系统变差？

多 Agent 系统的关键不是每个组件的平均知识量，而是联合错误分布。假设三个 Agent 单体都从 80% 提升到 84%，但共享记忆让它们在相同 16% 的 case 上一起错，那么多数票几乎失去价值。相反，三个 82% 的 Agent 若错误位置互补，系统 Majority 可能更可靠。

系统科学视角

我们关心的不是“memory 越多越好吗”，而是 memory architecture 如何改变 component dependence structure。

8. Diversity-Guard 不是“拒绝共享”，而是治理共享

- 保留 Local Quota：至少一部分 memory 永远来自本 Agent 自己的长期经历。
- Provenance：每条共享 experience 保留来源 Agent、task family、成功/失败与时间戳。
- Novelty Gate：与本地已有 memory 高度重复的经验不重复注入。
- Quality Gate：失败经验可以保留，但必须明确标注 failure / uncertainty，不能像成功示范一样直接复用。
- Cross-specialty Priority：优先补齐稀缺能力，而不是把大家最熟悉的东西再复制三遍。

9. 当前文献告诉了我们什么，没告诉我们什么？

2026 年的 DecentMem 直接指出集中式共享仓库可能带来协调开销、隐私问题并压缩 agent diversity，并提出 decentralized memory；Learning to Share 研究 selective cross-team memory reuse；ShareMem 说明跨用户共享能带来迁移，但 source quality 与 preference interference 会限制收益。另一方面，ACL 2026 的 Diversity Collapse 证明交互结构本身可以让多 Agent 过早收敛。

但是，这些工作并不能自动证明我们关心的完整链条已经被回答：Experience-Induced Heterogeneity → Memory Sharing → Error Re-coupling → Collective Gain Loss。第 15 讲的意义，是把这条链变成一个可以低成本验证的受控实验。

10. 真实实验最小闭环

- Step 1: 从第 14 讲选择一个已经出现稳定 specialization 的 checkpoint。
- Step 2: 冻结三个 agent bank，并 fork 四个 sharing protocol。
- Step 3: 每个 protocol 使用相同 memory budget、相同 held-out set。
- Step 4: 运行 1-2 个 sharing round 做 smoke test，再扩展到 4 轮。
- Step 5: 同时分析 transfer、erosion、memory/retrieval overlap、error ϕ 、wrong consensus 与 collective gain。
- Step 6: 人工精读出现“正确经验传播”和“共享经验诱发共错”的 trace。

11. 常见误区

- 误区：Full-Pool 准确率提高，所以共享一定好。——还要看相关错误是否增长。
- 误区：相关性升高就是共享导致。——必须从同一冻结起点 fork，并控制 memory budget。
- 误区：Diversity-Guard 越复杂越科学。——先做最小规则，避免方法堆叠。
- 误区：把失败经验删除掉。——失败 experience 可能非常有价值，但必须保留 provenance 与失败标签。
- 误区：只看最终 round。——长期 collapse 是动态过程，至少报告 round 0/1/2/4。

12. 本讲最低闭环能力

学完后至少要能做到

你应该能够：①解释 experience sharing 为什么既有 transfer benefit 又有 correlation cost；②设计冻结起点的 sharing ablation；③区分 memory overlap、retrieval overlap、behavior convergence 和 error re-coupling；④判断“更多共享”是否真的提高系统可靠性；⑤为第 16 讲的 shared-memory governance / selective consolidation 做准备。